CloudPulse AI

Intelligent Cloud Operations and Cost Optimization Platform

Complete Project Guide

Where to run the master build prompt, which tools to install, the full architecture, data model, API surface, DevOps and GitOps design, security, testing, documentation deliverables, phase-by-phase execution plan, cost notes and completion checklist.

Skills showcased: AWS and Azure, Python and TypeScript, FastAPI, PostgreSQL, Docker, Kubernetes, Terraform, GitHub Actions, Argo CD, machine learning, FinOps, observability and security.

Contents

1. About this guide and how to use it
2. Project at a glance
3. Where to use the master prompt
4. Tools and prerequisites
5. Technology stack and why each piece was chosen
6. System architecture and data flow
7. Repository structure
8. Database design
9. API design
10. Feature specifications
11. Cloud integrations (AWS and Azure)
12. DevOps: Docker, CI/CD, Terraform, Kubernetes, Argo CD
13. Observability and reliability
14. Security and threat model
15. Testing strategy
16. Required documentation deliverables
17. Phase-by-phase execution plan with ready prompts
18. Local setup and command reference
19. Cost estimation
20. Completion checklist
21. Tips, pitfalls and troubleshooting
22. Portfolio and interview presentation
23. Glossary
24. Official references

1. About this guide and how to use it

This document is a companion to your **Master Build Prompt** for CloudPulse AI. The prompt is a long instruction set written for an AI coding agent. It tells the agent to act as architect, engineer, DevOps specialist, security reviewer and technical writer, and to build a full working project. This guide explains what that prompt asks for, where you should paste it, which tools you must have installed, how to run it safely in stages, and what the finished project should look like.

What this guide contains

- **Where to use it:** a comparison of AI coding environments and the recommended way to run the prompt.
- **What tools to use:** every tool, runtime and service, with its purpose, whether it is required or optional, and a version check command.
- **Complete project information:** architecture, repository layout, database tables, API endpoints, features, DevOps design, security and testing.
- **Execution plan:** ten phases, each with a short prompt you can paste, the expected output and how to verify it.
- **Quality gates:** a completion checklist so that you only call the project finished when it truly is.

How the master prompt is structured

Prompt section	Topic	Where in this guide
1-2	Role, objective and project overview	Sections 1, 2
3	Required architecture	Sections 5, 6, 11
4	Functional requirements (4.1 to 4.11)	Section 10
5-6	API design and database model	Sections 8, 9
7	DevOps and GitOps	Section 12
8-9	Observability and security	Sections 13, 14
10	Testing strategy	Section 15
11	Required documentation	Section 16
12-15	Repository, developer experience, portfolio, cost	Sections 7, 18, 19, 22
16-18	Implementation phases, completion criteria, final report	Sections 17, 20

Note: This guide describes the intended design and the plan. It is not proof that the project exists or passes tests. Only the actual code, test output and deployment logs from your own run can show that. The master prompt itself requires the agent to never claim success for a command it did not run, and you should hold it to that.

2. Project at a glance

CloudPulse AI is a cloud operations and cost-intelligence platform. It ingests cloud cost data and operational events, stores them in PostgreSQL, analyses them with explainable anomaly detection and baseline forecasting, produces human-readable recommendations, and lets users manage alerts and incidents through a modern dashboard.

2.1 The problem it addresses

- Cloud bills grow in ways that are hard to attribute across services, regions and accounts.
- Sudden cost spikes or unusual usage often go unnoticed until the invoice arrives.
- Operational events (errors, deployments, throttling) live in separate tools from cost data, so cause and effect are hard to connect.
- Teams need a single place to see what changed, why it might have changed and what to do next.

2.2 Core capabilities

#	Capability	Short description
1	Cost ingestion	CSV upload with validation, normalisation, deduplication and run tracking
2	Event ingestion	API endpoint for operational events with search and filters
3	Dashboards	Totals, period-over-period change, trends, breakdowns by provider, service and region
4	Anomaly detection	Rolling statistics, MAD and optional Isolation Forest with explanations
5	Forecasting	Baseline method with backtesting, MAE and data-sufficiency warnings
6	Recommendations	Deterministic templates by default, optional LLM behind a strict schema
7	Alerts and incidents	Acknowledge, resolve, notes, linked events and anomalies
8	Search and export	Backend pagination, filters and CSV exports that honour permissions
9	Integrations	Optional AWS and Azure adapters plus a no-credentials demo mode
10	Operations	Metrics, health checks, audit events, CI/CD, Kubernetes and GitOps

2.3 Career skills the project demonstrates

Skill area	Where it appears in the project
AWS and Azure	Adapters for S3, Lambda, DynamoDB, CloudWatch, Cost Explorer, Azure Functions, Cosmos DB, Azure Monitor
Python backend	FastAPI, Pydantic, SQLAlchemy, Alembic, structured logging, background processing
JavaScript / TypeScript	Next.js, React, Tailwind CSS, Recharts, accessible components
Data and ML	Pandas, NumPy, scikit-learn, anomaly detection, forecasting, evaluation
DevOps	Docker, Compose, GitHub Actions, Terraform, Kubernetes, Helm, Argo CD
Security	Hashing, RBAC, rate limiting, CSRF, headers, audit logs, threat model
Reliability	Prometheus metrics, health and readiness, backups, failure simulation
Communication	Architecture documents, decision records, test plans, demo script

2.4 Design principles stated in the prompt

- **Local first, free first:** everything works with Docker Compose and synthetic data. Cloud and paid AI are optional.

- **Honesty:** synthetic data is always labelled; no invented accuracy, savings or citations; no feature described as tested unless it was.
- **Deterministic:** anomaly results are reproducible and are not randomised on page refresh.
- **Safe by default:** no hardcoded credentials, no destructive cloud actions, no unauthorised `terraform apply`.
- **Verifiable:** real tests, real command output, and a completion checklist.

3. Where to use the master prompt

The prompt must be pasted into an **agentic coding environment**: one that can read and write files in a project folder, run terminal commands, install packages and run tests. A normal chat window can only produce text. It cannot start Docker, run migrations or execute tests, so it cannot satisfy the prompt's requirement to build, run and verify.

3.1 Suitable environments

Environment	Type	Strengths	Considerations
Claude Code	Terminal agent (also desktop and IDE integrations)	Works directly in your repo, runs commands and tests, good for long multi-step builds	Requires an Anthropic account or API access; check current plan and usage limits
Cursor	AI-first code editor	Visual diffs, agent mode, easy to review changes	Agent usage subject to your plan; review terminal commands it proposes
GitHub Copilot (agent mode) in VS Code	IDE extension	Tight GitHub integration, familiar editor	Check which models and agent features your plan includes
OpenAI Codex CLI or similar CLI agents	Terminal agent	Scriptable, runs in your repo	Configure approval and sandbox settings carefully
Windsurf, Cline, Continue, Aider	Editor or CLI assistants	Flexible model choice, some support local or bring-your-own-key models	Quality depends heavily on the model you connect

Note: Product names, plans and features change often. Check each vendor's current documentation before choosing. Any environment with file access, a terminal and a strong coding model can work.

3.2 Recommended approach

1. Create an empty folder and run `git init` so every stage can be committed and rolled back.
2. Open that folder in your chosen agent (for example start the CLI in the folder, or open the folder in the editor).
3. Save the master prompt as `MASTER_PROMPT.md` in the folder so the agent can re-read it at any time.
4. Do **not** paste everything and walk away. Start with Phase 1 only (see Section 17) and review the plan.
5. Approve and run one phase at a time. After each phase, run the verification commands yourself and commit.
6. Keep the agent's permissions conservative: allow file edits and local commands, and require approval for network downloads, anything touching cloud accounts, and any destructive command.
7. At the end, run the completion checklist (Section 20) yourself on a clean checkout.

3.3 What each part of the project runs on

Part	Where it runs	Needs cloud account?
Frontend, backend, PostgreSQL, Redis	Your laptop via Docker Compose	No
Tests and linting	Your laptop and GitHub Actions	No
Kubernetes and Argo CD demo	Local kind or minikube cluster	No
Terraform for AWS	Validate and plan locally; apply only with your explicit approval	Yes, only for real deployment
AWS and Azure data adapters	Optional, server side, disabled by default	Yes, only if you enable them

Part	Where it runs	Needs cloud account?
Optional LLM explanations	Server-side call using a key from environment variables	Needs an AI provider key, optional

> **Important:** Never paste cloud keys, API keys or passwords into the prompt or into chat. Keep them in environment variables or a secret manager, and make sure `.env` is in `.gitignore`.

3.4 Hardware and account expectations

- A computer with at least 16 GB RAM is comfortable for Docker Compose plus a local Kubernetes cluster. 8 GB can work if you run only Compose.
- At least 15 GB of free disk space for images, dependencies and database volumes.
- A free GitHub account for the repository and Actions workflows.
- Optional: AWS and Azure accounts for the real integrations. Free tiers and credits have limits, so set budget alerts before you create anything.

4. Tools and prerequisites

Install and verify the following before starting. The version commands let you confirm each tool is available. Use currently supported versions and let the agent pin dependency versions in lock files.

4.1 Required for local development

Tool	Purpose	Check command
Git	Version control and GitHub workflow	git --version
Docker Desktop or Docker Engine + Compose plugin	Run frontend, backend, PostgreSQL, Redis	docker --version; docker compose version
Python 3.11 or newer (3.12 is a good choice)	Backend, analytics, tests	python3 --version
Node.js (current LTS) and npm	Next.js frontend and its tests	node --version; npm --version
Make	Task runner for make setup / up / test	make --version
An agentic coding tool	Builds the project from the prompt	See Section 3

Note: On Windows, use WSL 2 with an Ubuntu distribution so that Make, shell scripts and Docker behave as the documentation expects.

4.2 Required for quality and CI

Tool	Purpose
pytest, pytest-cov, httpx	Backend unit and API tests
Ruff (lint and format) and optionally mypy	Python style and type checks
ESLint, TypeScript compiler, Prettier	Frontend lint, type check, formatting
Vitest or Jest with React Testing Library	Frontend component tests
Playwright	End-to-end browser tests of the full workflow
pip-audit and npm audit (or Trivy, Dependabot)	Dependency and container vulnerability checks
GitHub Actions	CI pipeline running all of the above

4.3 Required only for the optional deployment tracks

Tool	Purpose	Needed when
Terraform (current stable)	Infrastructure as Code for AWS	Validating or planning infrastructure
tflint, checkov or tfsec	Terraform linting and security scanning	Reviewing IaC
kubectl	Talk to Kubernetes clusters	Kubernetes workflow
kind or minikube	Free local Kubernetes cluster	Local Kubernetes and Argo CD
Helm	Package and template Kubernetes manifests	Helm chart
Argo CD (installed in the cluster) and argocd CLI	GitOps continuous delivery	GitOps demo
AWS CLI v2	Authenticate to AWS (use SSO or roles)	AWS integration or deployment
Azure CLI	Authenticate to Azure	Azure integration
Prometheus and Grafana	Metrics and dashboards	Optional monitoring stack
Trivy or Grype	Container image scanning	Optional but recommended

4.4 Editors and helpers

- **VS Code** with Python, ESLint, Docker, Terraform and YAML extensions, or Cursor.
- **DBeaver or pgAdmin** to inspect PostgreSQL tables.
- **Postman, Bruno or curl** for manual API checks; the built-in FastAPI Swagger page at `/docs` also works.
- **Mermaid Live Editor or a Mermaid-capable Markdown viewer** to preview diagrams in the documentation.

5. Technology stack and why each piece was chosen

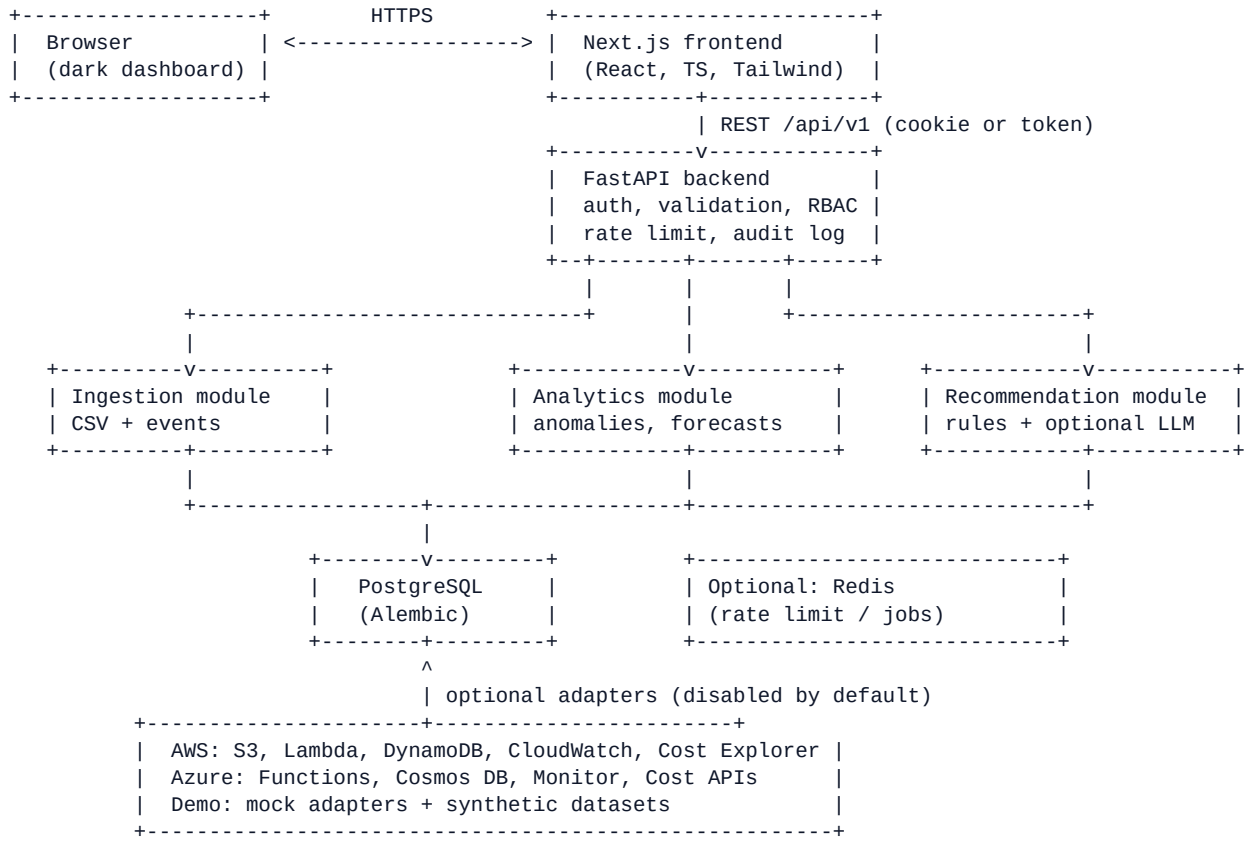
Layer	Choice	Why	Alternatives
Frontend framework	Next.js + React + TypeScript	Widely used, strong routing, type safety, good portfolio signal	Vite + React, Remix, SvelteKit
Styling	Tailwind CSS	Fast, consistent dark-mode design system	CSS Modules, Chakra UI, MUI
Charts	Recharts	React-native charts, simple API	Chart.js, ECharts, Visx
Backend	Python + FastAPI	Automatic OpenAPI docs, Pydantic validation, async support, same language as ML code	Django REST, Flask, NestJS
ORM and migrations	SQLAlchemy + Alembic	Mature, explicit schema migrations	SQLModel, Django ORM, Prisma
Database	PostgreSQL	Strong integrity, indexes, JSON support, time-series friendly queries	MySQL, TimescaleDB extension
Cache or jobs	Redis (optional)	Rate limiting and background jobs only if clearly needed	In-process limits, Celery, RQ, Arq
Data processing	Pandas, NumPy	Standard tabular transformation tools	Polars, DuckDB
ML	scikit-learn	Explainable, well-documented models such as Isolation Forest	statsmodels, Prophet (heavier)
Containers	Docker, Docker Compose	Reproducible local environment	Podman, devcontainers
CI/CD	GitHub Actions	Free for public repos, integrates with the repo	GitLab CI, Jenkins
IaC	Terraform	Industry standard, multi-cloud capable	AWS CDK, Pulumi, Bicep (Azure)
Orchestration	Kubernetes + Helm	Standard deployment target, shows platform skills	ECS, Azure Container Apps, Nomad
GitOps	Argo CD	Declarative, Git as source of truth, easy rollback	Flux CD
Observability	Prometheus + Grafana	Common open-source metrics stack	OpenTelemetry + vendor backends, Datadog
AI explanations	Provider abstraction, deterministic default	Works with no key; optional LLM behind strict schema	Local models via Ollama

5.1 Key trade-offs to record in DECISIONS.md

- **Monolith-style API versus many microservices:** a single FastAPI app with clear internal modules is simpler to run and test. The prompt lists `services/ingestion` and `services/analytics`; implement them as importable packages first and only split into separate processes if there is a real reason.
- **Session cookies versus JWT:** HttpOnly secure cookies with CSRF protection suit a same-site dashboard; bearer tokens suit separate clients. Choose one, document it, and test it.
- **Redis:** add it only if rate limiting or background jobs need shared state. Otherwise leave it out to reduce complexity.
- **Kubernetes optional:** keep Docker Compose as the simple path; Kubernetes and Argo CD are a documented advanced path.
- **Forecasting method:** start with seasonal-naive or moving-average baselines. They are easy to explain and to backtest honestly.

6. System architecture and data flow

6.1 High-level architecture



6.2 Component responsibilities

Component	Responsibility
Frontend (apps/web)	Pages, charts, forms, filters, error/empty/loading states, calls the API only; never holds secrets
API (apps/api)	Routing, authentication, authorization, validation, pagination, error format, metrics, audit events
Ingestion	Parse and validate CSV, normalise fields, compute row hashes for deduplication, record ingestion runs and errors; accept events
Analytics	Aggregations, anomaly detectors, forecasting with backtests, severity classification
Recommendations	Turn anomalies and cost shifts into structured findings with evidence and next steps
Cloud adapters	A common interface with AWS, Azure and mock implementations; connection tests and sync status
PostgreSQL	System of record for users, costs, events, anomalies, alerts, incidents, audit trail
Monitoring	Prometheus scrape config, Grafana dashboard JSON, alert rule examples

6.3 Cost CSV ingestion flow

1. User uploads a CSV on the Ingestion page. The frontend sends it to `POST /api/v1/ingestion/costs`.
2. The API checks authentication and role, then validates size limit, content type and extension.
3. An **ingestion run** row is created with status *running*.
4. The parser checks required columns and data types; invalid rows are collected with row number and reason.

5. Valid rows are normalised: timestamps to UTC dates, service names and regions to a canonical form, currency to ISO codes.
6. A deterministic hash of the natural key (date, provider, account, service, region, source) prevents duplicates when the same file is uploaded twice.
7. Rows are inserted in batches inside a transaction; counts of inserted, duplicate and rejected rows are stored.
8. The run is marked *succeeded*, *partial* or *failed* and a summary is returned to the user.
9. Optionally, anomaly detection is queued or triggered manually by the user.

6.4 Anomaly to alert flow

1. Detector reads cost history for a scope (provider, service, region) from the database.
2. It computes rolling baseline, MAD or Isolation Forest scores using configured thresholds.
3. Findings are classified by severity and enriched with explanation signals (for example: value, baseline, deviation, percent change).
4. A fingerprint of scope plus date window suppresses duplicates, so re-running detection does not create repeated alerts.
5. New anomalies produce alerts; the recommendation engine creates a linked recommendation.
6. Users acknowledge, annotate and resolve alerts; each change is stored and audited.

6.5 Mermaid diagram to include in the docs

```
sequenceDiagram
  participant U as User
  participant W as Web UI
  participant A as FastAPI
  participant D as PostgreSQL
  U->>W: Upload cost CSV
  W->>A: POST /api/v1/ingestion/costs
  A->>A: Validate, normalise, hash rows
  A->>D: Insert run + records (dedupe)
  A-->>W: Run summary (inserted, duplicate, rejected)
  U->>W: Click "Run detection"
  W->>A: POST /api/v1/anomalies/run
  A->>D: Read history, store anomalies, alerts
  A-->>W: Findings with explanations
```

7. Repository structure

Use a monorepo. Create directories only when they contain real content. The agent may simplify the layout, but should document why.

```
cloudpulse-ai/
  apps/
    web/                    Next.js frontend (pages, components, tests)
    api/                    FastAPI app (routers, models, schemas, core, migrations)
  services/
    ingestion/              CSV and event ingestion logic (package)
    analytics/              Anomaly detection, forecasting, recommendations
  packages/
    shared-schemas/         Shared API types / OpenAPI-generated client (optional)
  data/
    sample/                 Synthetic cost CSV and event JSON with a documented generator
  infra/
    docker/                 Dockerfiles and compose overrides
    terraform/              Modules and environments (dev, prod)
    kubernetes/             Plain manifests or kustomize overlays
    helm/                   Helm chart
  gitops/
    argocd/                 Argo CD Application and AppProject definitions
  monitoring/               Prometheus config, Grafana dashboards, alert rules
  scripts/                  Seed data, event generator, reset, chaos exercise
  tests/
    integration/            API and database integration tests
    e2e/                    Playwright end-to-end tests
  docs/                     All documentation deliverables
  .github/workflows/        CI, security scan, image build workflows
  docker-compose.yml
  Makefile
  .env.example
  .gitignore
  README.md
```

7.1 Backend internal layout (suggested)

```
apps/api/
  app/
    main.py                 App factory, middleware, router registration
    core/                   config, security, logging, rate limiting, errors
    db/                     session, base, repositories
    models/                 SQLAlchemy models
    schemas/                Pydantic request/response models
    routers/                auth, health, integrations, costs, events, anomalies,
                            forecasts, recommendations, alerts, incidents, exports, ingestion_runs
    adapters/               base.py, aws.py, azure.py, mock.py
  alembic/                  Migration environment and versions
  tests/                    unit and api tests
  pyproject.toml
```

7.2 Frontend internal layout (suggested)

```
apps/web/
  app/ or src/pages/        login, overview, costs, anomalies, recommendations,
                            events, alerts, incidents, integrations, ingestion, settings
  components/               charts, tables, filters, status badges, forms, layout
  lib/                      API client, auth helpers, formatters
  tests/                    component and form tests
```

8. Database design

The prompt requires normalised tables with primary and foreign keys, unique constraints, indexes, timestamps and Alembic migrations. No secrets are stored in application tables. The table below is a recommended starting design; the agent should finalise it in Phase 1 and document every field in `DATA_DICTIONARY.md`.

Table	Purpose	Key fields	Notable constraints and indexes
users	Accounts	id, email, password_hash, role_id, is_active, created_at, last_login_at	Unique email; hash only, never plaintext
roles	Role-based access	id, name (admin, readonly)	Unique name
cloud_integrations	Provider config without secrets	id, provider, mode (demo/aws/azure), enabled, config_json (non-secret), last_sync_at, status	Unique provider per workspace
ingestion_runs	Track each upload or sync	id, source_type, filename, status, rows_total, rows_inserted, rows_duplicate, rows_rejected, error_summary, started_at, finished_at, user_id	Index on started_at, status
cost_records	Normalised cost rows	id, usage_date, provider, account_identifier, service_name, region, usage_quantity, usage_unit, cost_amount, currency, source_type, is_synthetic, row_hash, ingestion_run_id	Unique row_hash; indexes on (usage_date), (provider, service_name), (region)
operational_events	Events history	id, external_event_id, provider, service, environment, severity, event_type, event_timestamp, message, correlation_id, metadata_json, is_synthetic	Unique (provider, external_event_id); indexes on timestamp, severity, correlation_id
anomalies	Detected cost anomalies	id, scope (provider, service, region), period_start, period_end, observed_value, expected_value, score, severity, method, signals_json, fingerprint, detected_at	Unique fingerprint; index on severity, detected_at
forecast_runs	Forecast results	id, scope, horizon_days, method, min_data_met, mae, params_json, results_json, created_at	Index on created_at
recommendations	Findings and next steps	id, anomaly_id, provider, service, summary, evidence_json, potential_impact, confidence_note, next_steps, requires_review, source (rule/llm), created_at	FK to anomalies
alerts	Actionable alerts	id, anomaly_id, title, severity, status (open/ack/resolved), provider, created_at, acknowledged_by, resolved_by, resolved_at, fingerprint	Unique fingerprint for suppression; index on status, severity
incidents	Grouped investigations	id, title, status, severity, opened_at, closed_at, owner_id	Index on status
incident_alerts / incident_events	Link tables	incident_id, alert_id or event_id	Composite primary keys
investigation_notes	Analyst notes	id, alert_id or incident_id, author_id, body, created_at	FK constraints, length limit
audit_events	Security trail	id, actor_id, action, entity_type, entity_id, ip_address, request_id, created_at, details_json	Append-only by convention; index on created_at, actor_id

8.1 Retention and deletion considerations

- Decide a retention period for cost records, events and audit logs (for example, raw events 90 days, aggregated cost 24 months) and document it.
- Provide a script or scheduled job to purge expired events; keep audit events longer than operational events.

- When a user is deactivated, keep audit references but remove or anonymise personal fields as policy requires.
- Back up PostgreSQL regularly and test restoration; document commands such as `pg_dump` and `pg_restore`.

8.2 ER diagram (Mermaid, include in DATABASE_DESIGN.md)

```
erDiagram
  USERS }o--|| ROLES : has
  INGESTION_RUNS ||--o{ COST_RECORDS : produces
  ANOMALIES ||--o{ ALERTS : raises
  ANOMALIES ||--o{ RECOMMENDATIONS : explains
  ALERTS ||--o{ INVESTIGATION_NOTES : has
  INCIDENTS ||--o{ INCIDENT_ALERTS : groups
  ALERTS ||--o{ INCIDENT_ALERTS : in
  INCIDENTS ||--o{ INCIDENT_EVENTS : groups
  OPERATIONAL_EVENTS ||--o{ INCIDENT_EVENTS : in
  USERS ||--o{ AUDIT_EVENTS : performs
```

9. API design

All endpoints live under `/api/v1`, are documented by OpenAPI, require authentication unless stated, and return a consistent error format. Writes require the administrator role; read-only users can view and export.

9.1 Conventions

Topic	Convention
Pagination	Query parameters page (default 1) and page_size (default 25, max 200). Response includes items, total, page, page_size
Sorting	sort_by and sort_dir with a whitelist of allowed fields and a stable default (for example newest first, then id)
Filtering	Explicit named parameters such as provider, service, region, severity, status, date_from, date_to, q (text search)
Errors	JSON body: {error: {code, message, details, request_id}} with correct status codes (400, 401, 403, 404, 409, 413, 422, 429, 500)
Rate limiting	Strict on login and upload; return 429 with Retry-After
Request IDs	Accept or generate X-Request-ID; include in logs and error bodies

9.2 Endpoint catalogue

Area	Method and path	Purpose	Role
Auth	POST /auth/login	Create session	Public (rate limited)
Auth	POST /auth/logout	End session	Any
Auth	GET /auth/me	Current user and role	Any
Health	GET /health	Liveness	Public
Health	GET /ready	Readiness incl. database check	Public
Metrics	GET /metrics	Prometheus metrics (restrict in production)	Internal
Integrations	GET /integrations	List providers, mode, status, missing config	Any
Integrations	PUT /integrations/{provider}	Select mode / enable (non-secret)	Admin
Integrations	POST /integrations/{provider}/test	Run connection test	Admin
Ingestion	POST /ingestion/costs	Upload cost CSV	Admin
Ingestion	GET /ingestion/runs	Run history with pagination	Any
Ingestion	GET /ingestion/runs/{id}	Run detail and errors	Any
Costs	GET /costs	Paginated, filtered cost records	Any
Costs	GET /costs/summary	Totals and period-over-period change	Any
Costs	GET /costs/timeseries	Daily or monthly series, grouped by provider, service or region	Any
Events	POST /events	Ingest event(s) with deduplication	Admin or API key
Events	GET /events	Search and filter events	Any
Anomalies	POST /anomalies/run	Run detection with config	Admin
Anomalies	GET /anomalies	List, filter and sort anomalies	Any
Forecasts	POST /forecasts/run	Generate forecast and backtest	Admin
Forecasts	GET /forecasts/latest	Latest forecast with warnings	Any

Area	Method and path	Purpose	Role
Recommendations	GET /recommendations	List and filter recommendations	Any
Alerts	GET /alerts	List with filters	Any
Alerts	GET /alerts/{id}	Detail with events and anomalies	Any
Alerts	POST /alerts/{id}/acknowledge	Acknowledge	Admin
Alerts	POST /alerts/{id}/resolve	Resolve	Admin
Alerts	POST /alerts/{id}/notes	Add investigation note	Admin
Incidents	GET /incidents; POST /incidents	List and create incidents	Any / Admin
Exports	GET /exports/{costs|anomalies|events|alerts|forecasts}.csv	CSV export honouring filters and role	Any
Exports	GET /exports/report	Period report stating synthetic vs provider data	Any

9.3 Example requests

```
# Login
curl -i -X POST http://localhost:8000/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@example.com","password":"<your-dev-password>"}'

# Upload synthetic cost CSV (cookie jar from login)
curl -b cookies.txt -F "file=@data/sample/synthetic_costs.csv" \
  http://localhost:8000/api/v1/ingestion/costs

# Ingest an operational event
curl -b cookies.txt -X POST http://localhost:8000/api/v1/events \
  -H "Content-Type: application/json" \
  -d '{"event_id":"evt-1001","provider":"aws","service":"Lambda","environment":"staging",
       "severity":"warning","event_type":"throttle","timestamp":"2025-03-02T10:15:00Z",
       "message":"Concurrent execution limit reached","correlation_id":"req-77",
       "metadata":{"region":"us-east-1"}}'
```

9.4 Example response shapes

```
// GET /api/v1/costs/summary?date_from=2025-02-01&date_to=2025-02-28
{
  "period": {"from": "2025-02-01", "to": "2025-02-28"},
  "total_cost": 18234.52, "currency": "USD",
  "previous_period_total": 16110.04, "change_percent": 13.19,
  "data_origin": "synthetic",
  "freshness": {"latest_usage_date": "2025-02-28", "last_ingestion_status": "succeeded"}
}

// Error
{"error": {"code": "validation_error", "message": "Invalid CSV",
           "details": [{"row": 14, "field": "cost_amount", "reason": "must be a number"}],
           "request_id": "9c2f..."}}
```

Note: The numbers above are illustrative formatting examples only, not measured results.

10. Feature specifications

10.1 Authentication and authorization

- Login and logout with passwords hashed by an established library (Argon2 or bcrypt through a maintained package).
- Two roles: administrator (read and write) and read-only. Authorization is enforced on the server for every protected route.
- Session handling uses HttpOnly, Secure, SameSite cookies with CSRF protection, or short-lived tokens with refresh handling. Expired sessions return 401 and the UI redirects to login.
- Rate limiting on login and upload endpoints. No public registration endpoint.
- Local bootstrap: a documented command creates the first admin using a password you supply through an environment variable or prompt. No default production credentials.

10.2 Cloud integrations page

- Shows each provider (AWS, Azure, Demo), connection status, last successful synchronisation and missing configuration fields.
- Mode selector: Demo, AWS, Azure. Connection test button runs a safe read-only check on the server.
- Setup instructions are displayed in the page and in documentation. The UI never requests root passwords or long-lived access keys.

10.3 Cost data ingestion

CSV schema (documented in `docs/` and in the sample data folder):

Column	Type	Rule
usage_date	date (ISO 8601)	Required; normalised to UTC date
provider	text	Required; one of aws, azure, other (normalised lowercase)
account_identifier	text	Required; account or subscription ID, no secrets
service_name	text	Required; normalised to canonical names
region	text	Required; normalised
usage_quantity	decimal	Required; non-negative
usage_unit	text	Required
cost_amount	decimal	Required; numeric
currency	ISO 4217 code	Required; uppercase, three letters
source_type	text	Required; synthetic, aws_cost_explorer, azure_cost_management, manual_csv

- Reject malformed rows with row number, field and reason; accept good rows in the same upload and report a partial result.
- Safe upload: size limit, extension and content checks, no execution of file content, streaming parse where practical.
- Duplicate-safe: uploading the same file twice inserts zero new rows the second time.
- Synthetic rows carry `is_synthetic = true` and the interface labels them clearly.

10.4 Operational events

Fields: event identifier, provider, service, environment, severity, event type, timestamp, message, correlation identifier, metadata. Include validation, deduplication on provider plus event identifier, searchable filters, and a script that generates test events. Treat message text as untrusted: escape it when rendering and never pass it unfiltered to an LLM or shell.

10.5 Dashboard

- Total cost for the selected date range and change versus the previous equivalent period.
- Daily and monthly trend charts; cost by provider, service and region.
- Recent events, unresolved alerts, anomaly counts, data freshness and ingestion health.
- Every chart has loading, empty and error states; date filters re-query the backend.
- A visible banner says *Demo / synthetic data* when the data is not provider-sourced.

10.6 Anomaly detection

Method	How it works	Strengths	Limitations
Rolling mean and standard deviation (z-score)	Compare each day to a trailing window	Simple, explainable	Sensitive to outliers and seasonality
Median absolute deviation (robust z-score)	Uses median and MAD	Robust to existing spikes	Needs enough history; may miss gradual drift
Isolation Forest (scikit-learn, fixed random_state)	Isolates unusual points in feature space	Handles several features at once	Less transparent; contamination parameter needs tuning

- Configurable thresholds (window size, z-score cut-off, minimum cost change).
- Severity levels (for example low, medium, high, critical) derived from deviation and cost impact.
- Each anomaly stores the signals that triggered it, such as observed value, baseline, deviation and percent change.
- A fingerprint suppresses repeated alerts. Detection is deterministic: same data and config give the same output.
- Unit tests use fixed synthetic normal and anomalous datasets; documentation lists false positives, limitations and tuning advice.

10.7 Cost forecasting

- Defined horizon (for example 7 or 30 days) and a minimum history requirement (for example at least 4 weeks of daily data).
- Baseline methods: moving average, seasonal naive (same weekday), optionally linear trend; choose and justify.
- Backtesting with a rolling origin and metrics such as MAE (and optionally MAPE or sMAPE with caution for small values).
- Warnings when data is too small or volatile. Historical and forecast values are drawn as separate series in the chart.
- No claims of accuracy or savings beyond the measured backtest on the documented dataset.

10.8 AI-assisted recommendations

Field	Content
Summary	One or two sentences describing the finding
Evidence	Measured facts: dates, values, baseline, linked anomaly and events
Potential impact	Estimated cost or risk, stated as an estimate
Confidence / uncertainty	Why the finding could be wrong; data-size caveats
Next steps	Human actions to investigate, never automatic infrastructure changes
Provider and service	Scope of the finding
Timestamp	When generated
Human review required	Boolean flag

- Default engine: deterministic rules and templates. Works fully offline.
- Optional LLM provider: configured by server-side environment variables; keys never reach the browser; secrets and credentials are never sent to the model.
- Model output is validated against a strict schema; failures fall back to the rule-based text.
- Provider data and event messages are untrusted input (guard against prompt injection). The LLM cannot execute commands.
- The UI labels AI-generated explanations separately from measured facts.

10.9 Alerts and incidents

- Alert list with filters by severity, status, provider and date; detail page with associated events and anomalies.
- Actions: acknowledge, resolve, add note. Changes persist and are audited.
- Incident history for grouped investigations. Repeated ingestion does not create unlimited duplicate alerts.

10.10 Search, filtering, exports

- Working search and filters for cost records, events, anomalies, recommendations, alerts and incidents, all with backend pagination and validated parameters.
- CSV exports for cost summaries, anomalies, events, alerts and forecasts; exports respect role and active filters.
- Generated period report that states whether data is synthetic or provider-sourced.
- Guard against CSV injection by neutralising cells that begin with =, +, - or @.

10.11 Pages to build

Page	Main content
Login	Form validation, error handling, session expiry message
Overview	KPI cards, trends, breakdowns, recent events, unresolved alerts, freshness
Cost analytics	Filters, tables, charts, forecast with separate historical and forecast series
Anomaly investigations	List, detail with signals, run detection with thresholds
Recommendations	Cards with evidence, impact, confidence and next steps
Event explorer	Search, filters, detail drawer, ingestion sample
Alerts and incidents	Lifecycle actions, notes, history
Ingestion	Upload, run history, error details

Page	Main content
Integrations	Provider status, connection tests, setup instructions
Settings and docs	Thresholds, data origin, links to documentation

11. Cloud integrations (AWS and Azure)

Integrations are implemented as modular adapters behind a common interface (for example `test_connection()`, `fetch_costs(start, end)`, `fetch_events(start, end)`). They are disabled by default. The mock adapter and synthetic datasets power all local functionality.

11.1 AWS mapping

Service	Role in CloudPulse AI	Access approach
Amazon S3	Store raw uploaded or exported cost files	IAM role or user with scoped bucket permissions; encryption and block public access
AWS Lambda	Event-driven ingestion (for example on S3 object creation)	Execution role with least privilege
Amazon DynamoDB	Optional operational event store	On-demand capacity for low cost; scoped table access
Amazon CloudWatch	Read metrics and logs; publish app metrics	Read-only policies for metrics and logs
AWS Cost Explorer API	Retrieve real cost data	Requires billing access and Cost Explorer enabled; API requests are charged per request, so cache results

11.2 Azure mapping

Service	Role in CloudPulse AI	Access approach
Azure Functions	Serverless ingestion triggered by storage or HTTP	Managed identity
Azure Cosmos DB	Optional operational record store	Role-based data access; serverless tier for low usage
Azure Monitor	Read metrics and logs	Reader roles through managed identity or service principal
Azure cost data	Cost Management APIs or exports	Cost Management Reader role on the scope; confirm the supported API in current Microsoft documentation

11.3 Credential rules

- No credentials in source control, Docker images, frontend code or database tables.
- Prefer IAM roles (AWS) and managed identities (Azure); for local testing use short-lived SSO credentials.
- Each integration reports which settings are missing and never prints secret values in logs.
- Document least-privilege policies and how to test each adapter with real credentials separately from the automated suite.

Important: Cloud billing APIs can incur charges and require specific account permissions. Set budgets and alerts in each cloud before enabling real integrations.

12. DevOps: Docker, CI/CD, Terraform, Kubernetes, Argo CD

12.1 Containers

- Multi-stage Dockerfiles for the API and the web app; slim base images; non-root user; HEALTHCHECK instructions.
- A `.dockerignore` that excludes .env files, node_modules, caches and git data.
- Pin base image tags and dependency versions; scan images with Trivy or similar.

12.2 Docker Compose

Service	Purpose	Notes
web	Next.js frontend	Depends on api; port 3000
api	FastAPI backend	Depends on db; port 8000; runs migrations on demand
db	PostgreSQL	Named volume for persistence; healthcheck
redis (optional)	Rate limit or jobs	Only if required
prometheus, grafana (optional profile)	Metrics and dashboards	Enable using a Compose profile

12.3 GitHub Actions workflows

Workflow	Jobs
ci.yml	Ruff lint and format check; pytest with coverage; ESLint; TypeScript check; frontend tests; API integration tests with a PostgreSQL service container
security.yml	pip-audit, npm audit, Trivy image scan, optionally CodeQL and Terraform scanning
build.yml	Build Docker images; publish to a registry only on main or tags once configured
terraform.yml	fmt, validate, tflint and plan on pull requests (no apply without approval)

- Set `permissions:` to the minimum (for example contents: read). Pin third-party actions to versions or commit SHAs.
- Use OpenID Connect to assume cloud roles rather than storing long-lived keys in repository secrets.
- Protect the main branch: require pull requests and passing checks.

12.4 Terraform (AWS)

- Reusable modules: networking, compute or container hosting, database, storage, IAM, monitoring, secrets.
- Environments (dev, prod) as separate variable sets or workspaces; remote state in S3 with locking, set up deliberately.
- Consistent tagging (project, environment, owner, cost-center) for FinOps reporting.
- Cost-conscious defaults: smallest practical instance classes, no NAT gateways unless needed, on-demand DynamoDB, log retention limits.
- Azure delivered as a documented alternative or extension, not provisioned alongside AWS by default.

```
terraform fmt -check -recursive
```

```
terraform init -backend=false
terraform validate
tflint
terraform plan -var-file=envs/dev.tfvars     # review only
# terraform apply   <- NEVER without your explicit approval
```

Important: The agent must never run terraform apply or create real cloud resources unless you explicitly authorise it. Always review the plan first, and know how to run terraform destroy.

12.5 Kubernetes and Helm

- Deployments, Services, ConfigMaps, Secret references (not plaintext), Ingress, probes, resource requests and limits.
- Security context: runAsNonRoot, read-only root filesystem where practical, dropped capabilities; NetworkPolicy examples.
- HorizontalPodAutoscaler guidance and notes on stateful services (use a managed or operator-managed PostgreSQL in production).
- Local workflow with kind or minikube: build images, load them into the cluster, install the chart, port-forward.

12.6 Argo CD and GitOps

- An Argo CD `Application` pointing to the Helm chart or manifests path in Git, with separate overlays or values per environment.
- Git is the source of desired state. Changes are made by pull request; Argo CD reconciles the cluster.
- Automated sync for dev; manual approval or protected branch for production. Self-heal and prune explained in the guide.
- Rollback by reverting the Git commit or using Argo CD history; secrets managed with Sealed Secrets, External Secrets or SOPS, never plaintext.

```
# CI/CD and GitOps flow (Mermaid)
flowchart LR
  Dev[Developer PR] --> CI[GitHub Actions: lint, test, scan, build]
  CI --> Reg[Container registry]
  CI --> Git[Update image tag in gitops repo path]
  Git --> Argo[Argo CD]
  Argo --> K8s[Kubernetes cluster]
  K8s --> Mon[Prometheus + Grafana]
```

13. Observability and reliability

Capability	Implementation
Structured logs	JSON logs with timestamp, level, request_id, route, status, latency; no secrets or personal data
Request IDs	Middleware generates or propagates X-Request-ID
Metrics	http_request_duration_seconds, http_requests_total by status, ingestion_runs_total by status, anomalies_detected_total, db_up
Health	/health for liveness; /ready checks the database and migration state
Dashboards	Grafana JSON for API latency, error rate, ingestion outcomes, anomaly counts
Alert thresholds	Examples: error rate over 5 percent for 5 minutes, p95 latency above target, ingestion failures, database down

13.1 Reliability documentation

- Common failure scenarios and recovery: database unavailable, migration failure, failed ingestion, disk full, cloud API throttling.
- Retry and timeout policy: explicit timeouts for outbound calls, bounded retries with exponential backoff and jitter, idempotent ingestion.
- Backup and restore: scripted `pg_dump` and restore steps, plus a tested restore drill in local development.
- Log retention and rotation guidance; list of service dependencies.

13.2 Controlled chaos exercise (local only)

1. Start the stack with Compose and confirm /ready returns healthy.
2. Stop the database container: verify the API returns safe errors and /ready reports not ready.
3. Start it again: verify automatic recovery and that no data was lost.
4. Upload the same CSV during and after the outage: verify idempotent behaviour.
5. Record observations in TEST_RESULTS.md with real timestamps and output.

> **Note:** Never run disruptive experiments against production or shared cloud resources.

14. Security and threat model

14.1 Baseline controls

Control	Implementation
Input validation	Pydantic models, strict CSV validation, allow-listed sort and filter fields
SQL injection	SQLAlchemy parameterised queries only
AuthN and AuthZ	Hashed passwords, RBAC enforced server side, tests for forbidden access
Sessions	HttpOnly, Secure, SameSite cookies; expiry handling; CSRF protection when cookies are used
Transport and headers	TLS at the edge; CSP, X-Content-Type-Options, X-Frame-Options or frame-ancestors, Referrer-Policy
CORS	Explicit origin allow-list; no wildcard with credentials
Rate limiting	Login, upload and event ingestion endpoints
File upload	Size cap, type checks, parse as data only, temp-file cleanup
Secrets	Environment variables or secret managers; .env.example with placeholders only
Dependencies	Lock files, pip-audit, npm audit, Dependabot, image scanning
Audit logging	Login, ingestion, role changes, alert state changes, exports
Containers	Non-root, minimal images, no secrets baked in
Cloud	Least-privilege IAM, managed identities, encrypted storage, budget alerts

14.2 Threat model outline (THREAT_MODEL.md)

Asset	Threat	Mitigation
User credentials	Brute force, credential stuffing	Rate limiting, strong hashing, lockout or backoff, audit logs
Session	Theft, CSRF, fixation	HttpOnly cookies, CSRF tokens, rotation on login, short expiry
Cost and event data	Unauthorised read or export	RBAC, export checks, audit trail
Upload endpoint	Malicious or oversized files, CSV injection	Limits, validation, output sanitisation
LLM integration	Prompt injection through event messages, data leakage	Treat input as untrusted, strict output schema, no secrets sent, no tool execution
Cloud credentials	Leak or over-permissioned roles	Roles and managed identities, least privilege, no keys in code
CI/CD pipeline	Compromised action or token	Pinned actions, minimal permissions, OIDC, protected branches
Kubernetes	Privilege escalation, lateral movement	Security contexts, NetworkPolicy, namespace separation, RBAC

Trust boundaries to diagram: browser to frontend, frontend to API, API to database, API to cloud providers, API to LLM provider, and CI/CD to registry and cluster.

> **Note:** Do not claim the system is completely secure. State residual risks and what has not been independently reviewed.

15. Testing strategy

Level	Scope	Tooling
Backend unit	CSV validation, normalisation, hashing and deduplication, anomaly detectors, forecast metrics, recommendation rules	pytest
Backend API	Auth, RBAC, pagination, filters, error format, alert lifecycle, exports	pytest + httpx / TestClient
Database integration	Migrations apply cleanly, constraints enforced, transactions	pytest with PostgreSQL service or testcontainers
Frontend component	Charts states (loading, empty, error), forms, filters	Vitest or Jest + React Testing Library
Frontend flows	Login, session expiry, dashboard date filters	Testing Library, MSW for API mocks
End to end	Complete user journey in a browser against the running stack	Playwright
Security checks	Dependency and image scans, IaC scans	pip-audit, npm audit, Trivy, checkov
Resilience	Database restart, repeated upload	Scripted chaos exercise

15.1 End-to-end scenario (from the prompt)

1. Start the application with Docker Compose.
2. Authenticate as the bootstrapped administrator.
3. Upload the supplied synthetic cost CSV.
4. Verify ingestion results (inserted, duplicate, rejected counts).
5. View cost summaries on the dashboard.
6. Run anomaly detection and review the findings.
7. Generate a forecast and read its warnings and MAE.
8. Open a recommendation and check its evidence.
9. Ingest an operational event.
10. Acknowledge and resolve an alert, and add a note.
11. Export a CSV and the period report.
12. Restart services and confirm the data persists.

15.2 Rules for recording results

- Use deterministic fixtures and fixed random seeds.
- Paste real command output (pass counts, failures, coverage) into TEST_RESULTS.md with the date and environment.
- List tests that could not run (for example real AWS or Azure tests) and why.
- Fix failing tests before reporting completion; do not delete tests to make a suite pass.

16. Required documentation deliverables

All documentation must be real files in the repository, not only chat output. Docs must match the implemented code, and any planned or optional feature must be labelled as such.

File	Contents
README.md	Summary, features, screenshots, architecture overview, stack, prerequisites, install, env setup, migrations, seed, start/stop, tests, troubleshooting, deployment overview, limitations
docs/PROJECT_DOCUMENTATION.md	The 30-section project report (see list below)
docs/ARCHITECTURE.md	Components, data flow, Mermaid diagrams
docs/API_REFERENCE.md	Endpoints, schemas, status codes, examples
docs/DATABASE_DESIGN.md	Schema rationale, ER diagram, constraints, retention
docs/DATA_DICTIONARY.md	Every table, field, type and relationship
docs/DEPLOYMENT_GUIDE.md	Local, optional cloud, and production instructions; teardown
docs/GITOPS_GUIDE.md	Argo CD setup, reconciliation, rollback, secrets
docs/SECURITY.md	Controls, configuration, reporting vulnerabilities
docs/THREAT_MODEL.md	Assets, boundaries, threats, mitigations
docs/TEST_PLAN.md	Strategy, scope, environments
docs/TEST_RESULTS.md	Actual executed results and gaps
docs/TROUBLESHOOTING.md	Common errors with fixes
docs/COST_ESTIMATION.md	Assumptions and estimated cloud costs
docs/DEMO_SCRIPT.md	A 5 to 10 minute guided demo for recruiters
docs/DECISIONS.md	Architecture decision records
docs/CONTRIBUTING.md	Workflow, coding standards, commit conventions

16.1 Project report outline (PROJECT_DOCUMENTATION.md)

Title page; executive summary; problem statement; motivation and objectives; scope and exclusions; users and use cases; functional requirements; non-functional requirements; feasibility and trade-offs; system architecture; component responsibilities; data flow; database design; API design; ingestion pipeline; machine learning methodology; forecasting methodology and evaluation; AI recommendation design; security and threat model; CI/CD and GitOps; infrastructure deployment; observability and reliability; testing strategy and actual results; setup and execution guide; sample demonstrations; known limitations; cost considerations; future improvements; conclusion; references to official documentation.

16.2 Required Mermaid diagrams

- System architecture
- Data ingestion sequence
- Authentication flow
- Anomaly detection flow
- CI/CD and GitOps deployment
- Database entity relationships

17. Phase-by-phase execution plan with ready prompts

Run the phases in order. For each phase, paste the short prompt after loading `MASTER_PROMPT.md` into the agent's context, then run the verification steps yourself and commit with Git.

Phase 1 - Requirements and design

Paste this prompt:

> Read MASTER_PROMPT.md. Inspect the workspace. Produce docs/REQUIREMENTS_CHECKLIST.md, a final architecture, the database schema as a draft, and a list of assumptions and open questions. Do not write application code yet.

Expected result: Checklist covers every section of the prompt; schema reviewed; assumptions listed.

Verify: Read the files, ask for changes, then git commit.

Phase 2 - Backend foundation

Paste this prompt:

> Implement Phase 2: configuration, SQLAlchemy models, Alembic migrations, authentication with roles, health and readiness endpoints, error format, structured logging, sample-data generator, and initial pytest tests. Run the tests and report real output.

Expected result: API starts, migrations apply to an empty database, login works, auth tests pass.

Verify: docker compose up db; alembic upgrade head; pytest; open /docs.

Phase 3 - Ingestion and analytics

Paste this prompt:

> Implement Phase 3: CSV upload with validation and deduplication, ingestion runs, cost aggregation endpoints, event ingestion, deterministic anomaly detection, baseline forecasting with backtesting, and rule-based recommendations. Add unit tests with fixed synthetic datasets.

Expected result: Same file uploaded twice inserts no duplicates; anomalies are reproducible; forecast warns on small data.

Verify: pytest; curl the endpoints; compare two detection runs for identical output.

Phase 4 - Frontend

Paste this prompt:

> Implement Phase 4: all pages listed in the prompt using real API data, with loading, empty and error states, filters, forms, alert actions and exports. Add component tests and make the data-origin banner visible.

Expected result: Every chart is driven by the API; date filters change results; forms validate.

Verify: npm run lint; npm run typecheck; npm test; use the app in a browser.

Phase 5 - Cloud integrations

Paste this prompt:

> Implement Phase 5: adapter interface, mock adapter, optional AWS and Azure adapters, integrations page and connection tests. Everything must remain disabled by default and work without credentials. Use mocks in automated tests.

Expected result: Demo mode works with no credentials; missing configuration is reported clearly.

Verify: Run tests; open the Integrations page; confirm no secrets in code.

Phase 6 - DevOps and deployment

Paste this prompt:

> Implement Phase 6: Dockerfiles, docker-compose.yml, Makefile, GitHub Actions workflows, Terraform modules for AWS, Kubernetes manifests or Helm chart, and Argo CD application. Validate only; do not apply anything to a real account.

Expected result: docker compose up works; terraform validate passes; helm lint and kubectl dry-run pass.

Verify: make up; terraform validate; helm lint; kind cluster test if desired.

Phase 7 - Security and observability

Paste this prompt:

> Implement Phase 7: security headers, CORS, CSRF, rate limiting, audit events, Prometheus metrics, request IDs, monitoring configs, threat model and recovery documentation.

Expected result: Security tests pass; /metrics exposes metrics; audit rows are written.

Verify: curl -I to inspect headers; run security tests; view metrics.

Phase 8 - Testing

Paste this prompt:

> Implement Phase 8: run all backend and frontend tests, write and run the Playwright end-to-end workflow, run the failure simulation, fix defects, and record actual results in docs/TEST_RESULTS.md.

Expected result: End-to-end scenario passes, including data persistence after restart.

Verify: make test; npx playwright test; compare TEST_RESULTS.md to real output.

Phase 9 - Documentation

Paste this prompt:

> Implement Phase 9: write every documentation file listed in the prompt, include the Mermaid diagrams, and verify that each command, path and environment variable in the docs exists and works.

Expected result: No document references a feature that is not implemented.

Verify: Follow the README from scratch in a clean clone.

Phase 10 - Final review

Paste this prompt:

> Implement Phase 10: verify the project from a clean checkout, confirm the demo works without cloud credentials, scan for committed secrets, check the completion criteria in section 17, and produce the final status report with honest limitations.

Expected result: Every completion criterion is marked done, partial or not done with evidence.

Verify: git clone to a new folder; follow README; run gitleaks or a similar scan.

17.1 Useful follow-up prompts

- "Show me the exact commands you ran and their output for the last step."
- "List every requirement from section X of MASTER_PROMPT.md and mark it implemented, partial or missing, with file paths."
- "Find any place where documentation claims something that is not implemented or not tested."
- "Run the tests again and fix only the failing ones without weakening assertions."
- "Review the code for security issues against docs/THREAT_MODEL.md."

18. Local setup and command reference

These are the commands the prompt asks the project to support. They are targets to verify after the build; confirm each exists in the Makefile that the agent generates.

```
# 1. First-time setup
git clone <your-repo-url> cloudpulse-ai && cd cloudpulse-ai
cp .env.example .env          # edit values; never commit .env
make setup                    # install dependencies, pre-commit hooks

# 2. Start services
make up                       # docker compose up -d (web, api, db)
make migrate                  # alembic upgrade head
make seed                     # synthetic data + bootstrap admin (uses env-provided password)

# 3. Open the app
#   Frontend:  http://localhost:3000
#   API docs:  http://localhost:8000/docs

# 4. Quality
make lint                     # ruff, eslint, tsc
make test                     # pytest, frontend tests (and e2e if configured)

# 5. Stop and reset
make down                     # stop containers
make reset                    # remove volumes and recreate dev data (destructive, dev only)
```

18.1 Environment variables (example names)

Variable	Purpose	Secret?
DATABASE_URL	PostgreSQL connection string	Yes
SECRET_KEY	Signing key for sessions or tokens	Yes
ADMIN_EMAIL / ADMIN_PASSWORD	Bootstrap admin for local development only	Yes
CORS_ORIGINS	Allowed frontend origins	No
APP_ENV	development, test or production	No
DEFAULT_DATA_MODE	demo, aws or azure	No
AWS_REGION, AWS_ROLE_ARN	Optional AWS integration (use roles; no static keys)	Role ARN not secret
AZURE_TENANT_ID, AZURE_CLIENT_ID	Optional Azure integration with managed identity	Partly
LLM_PROVIDER, LLM_API_KEY	Optional AI explanations, server side only	Key is secret
REDIS_URL	Only if Redis is used	Maybe

18.2 Kubernetes local workflow (optional)

```
kind create cluster --name cloudpulse
docker build -t cloudpulse-api:dev apps/api && docker build -t cloudpulse-web:dev apps/web
kind load docker-image cloudpulse-api:dev cloudpulse-web:dev --name cloudpulse
helm install cloudpulse infra/helm/cloudpulse -n cloudpulse --create-namespace
kubectl -n cloudpulse get pods
kubectl -n cloudpulse port-forward svc/cloudpulse-web 3000:80

# Argo CD (local demo)
kubectl create namespace argocd
kubectl apply -n argocd -f <official Argo CD install manifest URL from its docs>
kubectl apply -f gitops/argocd/application.yaml
```

18.3 Database operations

```
# Backup
docker compose exec db pg_dump -U <user> -d cloudpulse -Fc -f /tmp/backup.dump
# Restore into a fresh database
docker compose exec db pg_restore -U <user> -d cloudpulse_restore /tmp/backup.dump
# New migration after model change
docker compose exec api alembic revision --autogenerate -m "describe change"
```

19. Cost estimation

Local development with Docker Compose and synthetic data costs nothing beyond your own hardware and electricity. Cloud and AI costs depend on region, usage and current pricing, so treat the figures below as **planning categories**, not quotes. Use the AWS Pricing Calculator and Azure Pricing Calculator to produce real estimates for your configuration.

Track	What you pay for	Expected order of magnitude	How to control it
Local development	Nothing (own computer)	Free	Use Compose profiles; stop containers when idle
GitHub	Public repo and Actions on standard runners	Typically free within current free limits	Check current GitHub plan limits
Small AWS deployment	Compute or containers, database, storage, logs, data transfer	Can range from a few dollars to tens of dollars per month for a minimal setup; managed Kubernetes and NAT gateways raise it noticeably	Budgets and alarms, small instances, destroy when not demoing
Cost Explorer API	Per-request charges for API calls	Small if cached	Cache results; avoid frequent polling
Azure alternative	Container hosting, Cosmos DB, storage, monitoring	Similar planning range	Serverless tiers, budgets
Optional LLM	Tokens used for explanations	Depends on provider and volume	Cache outputs; limit calls; keep deterministic default

- Create a budget and email alerts in AWS Budgets or Azure Cost Management before deploying anything.
- Tag every resource so costs are attributable to the project.
- Document teardown commands (`terraform destroy`, deleting buckets and logs) and verify nothing is left running.
- Do not promise that hosting will be free; free tiers expire and have limits.

20. Completion checklist

Mark an item done only with evidence (command output, file path or screenshot).

Done	Item
[ ]	Frontend and backend run locally from a clean checkout.
[ ]	Database migrations apply to an empty database.
[ ]	Authentication and authorization are implemented and tested (admin and read-only).
[ ]	Synthetic data ingestion works; uploading the same file twice creates no duplicates.
[ ]	Dashboard metrics are calculated from stored data; date filters work; loading, empty and error states exist.
[ ]	Anomaly detection produces identical results on repeated runs with the same data and configuration.
[ ]	Forecasting shows its evaluation metric and limitation warnings.
[ ]	Recommendations trace back to real anomalies and cost data.
[ ]	Alerts and incidents can be acknowledged, resolved and annotated; changes persist after restart.
[ ]	CSV exports and the period report work and respect filters and roles.
[ ]	OpenAPI documentation is available and main endpoints are documented.
[ ]	Tests have been executed and real results recorded in TEST_RESULTS.md.
[ ]	GitHub Actions workflows, Dockerfiles, Compose, Terraform, Kubernetes or Helm and Argo CD files are present and validated.
[ ]	Optional AWS and Azure integrations clearly report whether they are configured; demo mode works without credentials.
[ ]	No secrets are committed (scan the repository and history).
[ ]	All required documentation files exist and match the implementation.
[ ]	No feature is described as tested or deployed unless it actually was.

21. Tips, pitfalls and troubleshooting

21.1 Working effectively with an AI coding agent

- **One phase at a time.** A single enormous request often exceeds what the agent can finish and verify. Stage the work.
- **Commit often.** Tag each phase so you can roll back when the agent goes in a wrong direction.
- **Keep the prompt in the repo.** Agents lose earlier context in long sessions; ask them to re-read MASTER_PROMPT.md and the checklist at the start of each phase.
- **Verify yourself.** Run the tests and open the app. Agents can sound confident when something is broken.
- **Review risky commands.** Keep approval on for deletions, network installs, cloud CLIs and anything with sudo.
- **Ask for evidence.** Request exact commands and output, then compare with what is in TEST_RESULTS.md.
- **Control scope.** Say explicitly: no real cloud calls, no terraform apply, no committing secrets.

21.2 Common problems

Symptom	Likely cause	Fix
Port already in use (3000, 8000, 5432)	Another local service	Stop it or change the port mapping in docker-compose.yml
API cannot connect to the database	Database not healthy yet or wrong DATABASE_URL	Check docker compose ps and logs; use the service name db as host inside Compose
Alembic reports a multiple heads or missing revision error	Conflicting migrations	Run alembic heads; merge or fix revision order; reset dev DB if safe
Frontend gets CORS or 401 errors	Origin not allowed or cookie settings mismatch	Set CORS_ORIGINS; check SameSite and Secure flags for http on localhost
Upload rejected as too large or invalid	Size limit or schema mismatch	Check the error body; compare with the documented CSV schema
Second upload creates duplicates	Row hash omits a key field	Review hashing fields; add a unit test and unique constraint
Anomaly results change between runs	Unseeded randomness or unstable sorting	Set random_state; sort inputs deterministically
Docker build is slow or huge	Missing .dockerignore or single-stage build	Use multi-stage builds, slim images and layer caching
Windows path or line ending problems	Not using WSL 2	Run everything inside WSL 2; configure git autocrlf appropriately
Tests pass locally but fail in CI	Different versions or missing service	Pin versions; add PostgreSQL service container; mirror local env variables
Agent claims success without evidence	Overconfident summary	Ask for command output; run the command yourself

22. Portfolio and interview presentation

22.1 Repository polish

- A clear README with a short animated GIF or screenshots of the dashboard, anomaly page and alert flow.
- Architecture diagram at the top, a quick-start section that works in under ten minutes, and a limitations section.
- Badges for CI status, license and test results (only if they are real).
- Release tag and a short changelog.

22.2 Five-minute demo script

1. State the problem: cost spikes and scattered events in multi-cloud environments.
2. Log in and show the Overview with the synthetic-data banner.
3. Upload the sample CSV; show the ingestion run and duplicate protection by uploading it again.
4. Open Cost analytics; change the date range; show provider, service and region breakdowns.
5. Run anomaly detection; open one anomaly and explain the signals and severity.
6. Show the recommendation with evidence and uncertainty, and note it is rule-based by default.
7. Ingest an event, open the alert, acknowledge, add a note and resolve.
8. Show the Integrations page, mock versus AWS or Azure status, and explain the credential model.
9. Briefly show CI, Docker Compose, Terraform plan and the Argo CD application.
10. Close with limitations and next steps.

22.3 Questions to prepare for

- Why FastAPI and PostgreSQL? Why a deterministic baseline before machine learning?
- How do you prevent duplicate ingestion and duplicate alerts?
- How would you scale ingestion to millions of rows? (Partitioning, batch loading, queues, columnar stores.)
- How do you secure the LLM integration against prompt injection?
- What are the limitations of your anomaly and forecast methods, and how did you evaluate them?
- How does GitOps change deployment and rollback compared with pushing from CI?
- How would you reduce cloud cost for this application itself?

22.4 How to describe it on a resume

Describe only what you built and verified. Example structure: *Built a full-stack cloud cost and operations platform with FastAPI, PostgreSQL, Next.js and Docker; implemented CSV ingestion with idempotent deduplication, explainable anomaly detection and baseline forecasting with backtesting; automated CI with GitHub Actions and defined Terraform, Kubernetes and Argo CD deployment configuration.* Add measured facts (test counts, coverage) only from your own recorded results.

23. Glossary

Term	Meaning
FinOps	Practice of managing and optimising cloud spending through collaboration between engineering, finance and operations
Anomaly	A data point or period that deviates significantly from expected behaviour
MAD	Median absolute deviation, a robust measure of spread
Isolation Forest	Tree-based unsupervised algorithm that isolates outliers
MAE	Mean absolute error, average absolute difference between forecast and actual values
Backtesting	Evaluating a forecasting method on past data by simulating forecasts at earlier points in time
Idempotent	An operation that yields the same result when repeated, such as re-uploading the same file
RBAC	Role-based access control
IaC	Infrastructure as Code, defining infrastructure in version-controlled files
GitOps	Operating infrastructure and applications by treating Git as the source of truth for desired state
Argo CD	Kubernetes controller that continuously reconciles cluster state with Git
Helm	Package manager and templating system for Kubernetes
OIDC	OpenID Connect, used by CI systems to obtain short-lived cloud credentials without stored keys
CSRF	Cross-site request forgery, an attack on cookie-authenticated sessions
Prompt injection	Malicious text that tries to change an LLM's behaviour
Managed identity	Azure-provided identity for resources that removes the need for stored credentials

24. Official references

Use official vendor documentation as the source of truth and check current versions before relying on any command or price.

Topic	Where to look
FastAPI	fastapi.tiangolo.com
Pydantic	docs.pydantic.dev
SQLAlchemy and Alembic	docs.sqlalchemy.org; alembic.sqlalchemy.org
PostgreSQL	postgresql.org/docs
Next.js and React	nextjs.org/docs; react.dev
Tailwind CSS	tailwindcss.com/docs
Recharts	recharts.org
pandas, NumPy, scikit-learn	pandas.pydata.org/docs; numpy.org/doc; scikit-learn.org/stable
pytest and Playwright	docs.pytest.org; playwright.dev
Docker and Compose	docs.docker.com
GitHub Actions	docs.github.com/actions
Terraform	developer.hashicorp.com/terraform
Kubernetes and kind	kubernetes.io/docs; kind.sigs.k8s.io
Helm	helm.sh/docs
Argo CD	argo-cd.readthedocs.io
Prometheus and Grafana	prometheus.io/docs; grafana.com/docs
AWS (S3, Lambda, DynamoDB, CloudWatch, Cost Explorer, IAM)	docs.aws.amazon.com; AWS Pricing Calculator
Azure (Functions, Cosmos DB, Monitor, Cost Management)	learn.microsoft.com/azure; Azure Pricing Calculator
Security guidance	owasp.org (Top 10, ASVS, Cheat Sheet Series, LLM Top 10)
FinOps	finops.org

Note: Citations in the final project report must point to real pages that support the specific statement made. Do not invent references. If you cannot verify a claim, remove it.